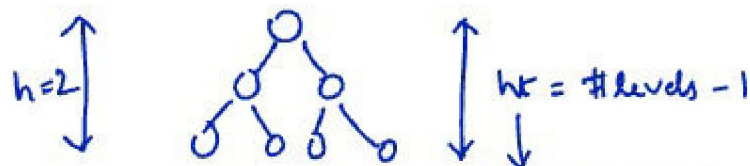


Binary tree: A tree where each node has a max of 2 children.

Perfect Binary tree: All the levels of the binary tree are full.



length of the longest path from the root to any of its leaves.

For a perfect binary tree

$$\#nodes = \underbrace{2^0 + 2^1 + 2^2 + \dots + 2^h}_{\#terms = h+1}$$

$$= \frac{2^{h+1} - 1}{2 - 1} = 2^{h+1} - 1$$

$$\boxed{n = 2^{h+1} - 1}$$

$$\Rightarrow 2^{h+1} = n + 1 \rightarrow \log_2(n+1) = h+1$$

$$\Rightarrow \log_2 2^{h+1} = \log_2(n+1)$$

$$\Rightarrow (h+1) \log_2 2 = \log_2(n+1)$$

$$\Rightarrow (h+1) = \log_2(n+1)$$

$$\Rightarrow \boxed{h = \log_2(n+1) - 1}$$

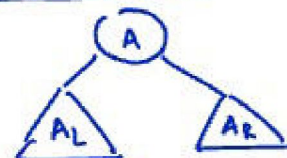
$$\boxed{h = \Theta(\log n)}$$

$\log_b b^n = n$
$\log_b b = 1$
$\log_b 1 = 0$

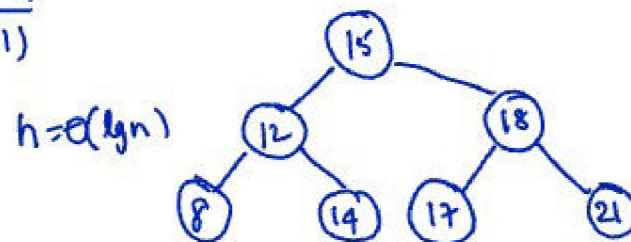
$$GP \text{ sum} = a \frac{r^n - 1}{r - 1}$$

↑
#terms

Binary Search trees

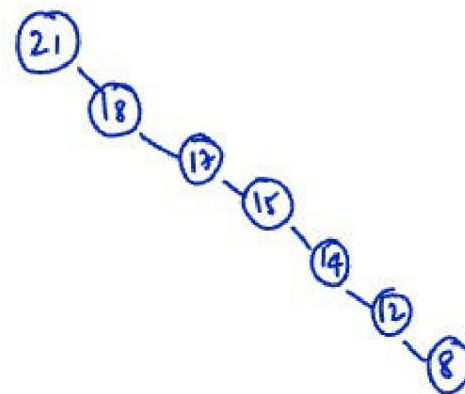


all vals in $A_L < A$'s val
all vals in $A_R > A$'s val



$$h = \Theta(\log n)$$

$$h = \Theta(n)$$



Heaps

- Complete Binary trees

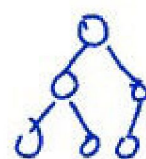
1. Balanced

$$h = \Theta(\log n)$$

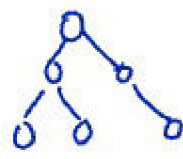
2. stored using arrays

Complete Binary tree:

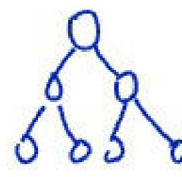
- All levels are full except possibly the last level
- Last level must be filled from left to right



Complete! ✓

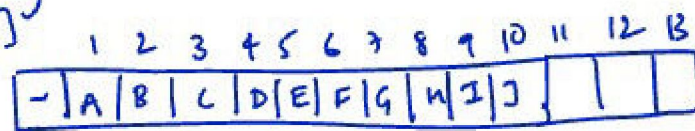


Complete! ✗



Complete! ✓

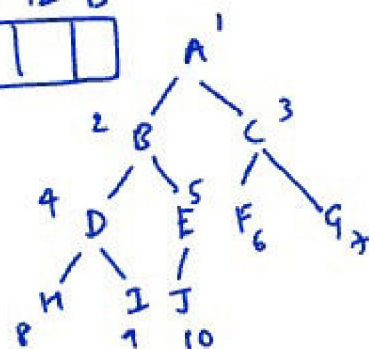
int[] → 4n bytes



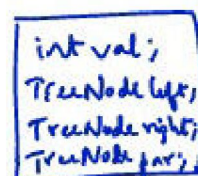
$$\text{index of parent}(i) = \lfloor i/2 \rfloor$$

$$\text{left child}(i) = 2*i$$

$$\text{right child}(i) = 2*i + 1$$



TreeNode



4 bytes

8 bytes

8 bytes

padding: 4 bytes

total size: 24 bytes

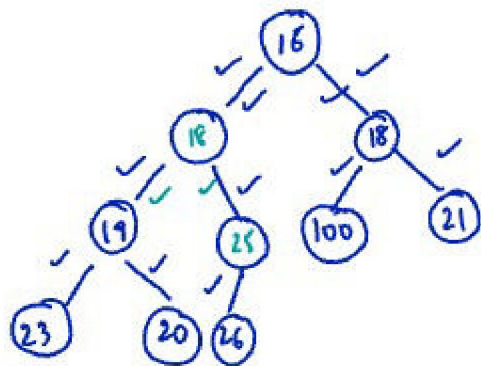
24n bytes for n nodes

• swim()

• sink()

• Min heaps:

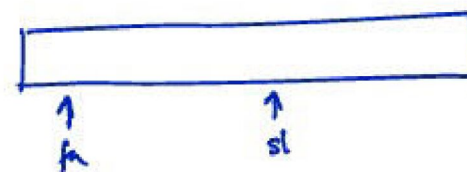
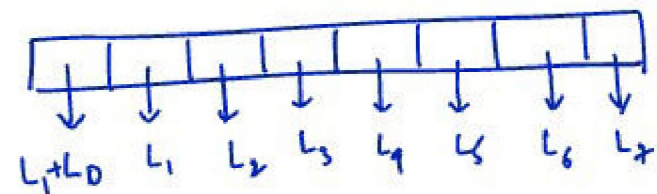
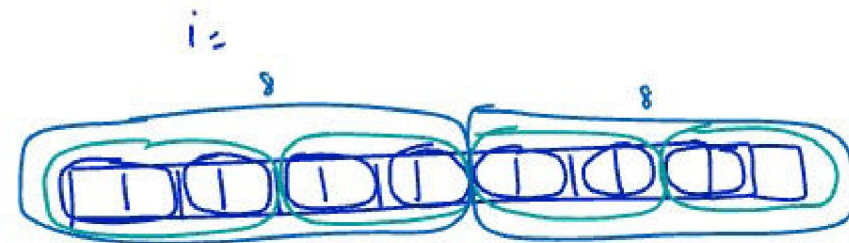
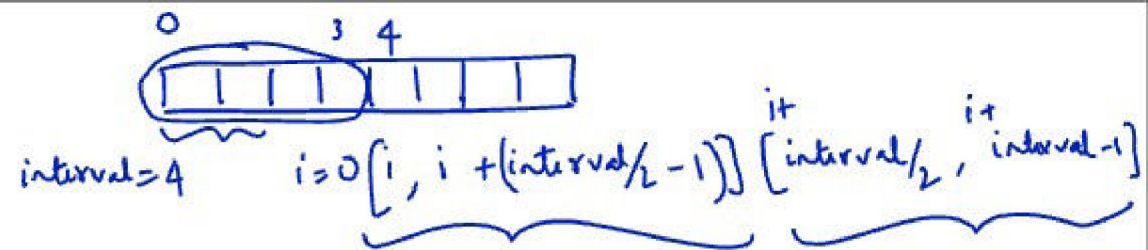
• Property: For every node, its value should be $\geq$ the parent's value



Invariants

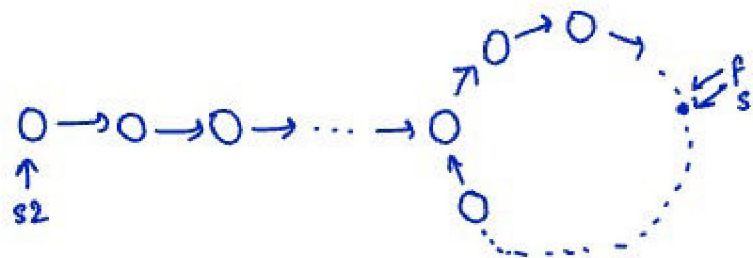
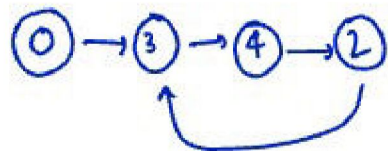
1. Heap property
2. Complete Binary Tree ✓

15



means $\checkmark$ 0 1 2 3 4
[3, 1, 3, 4, 2]

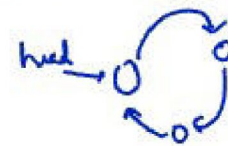
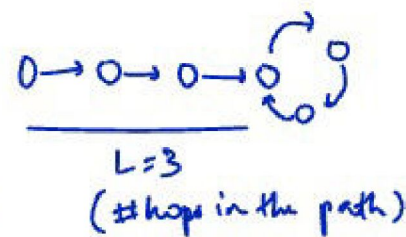
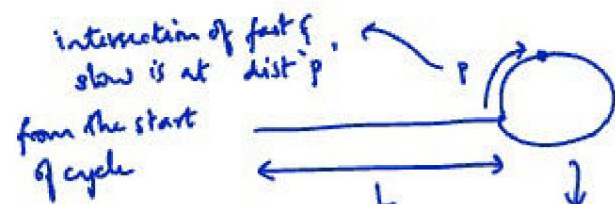
$n+1$ integers
[1, n]



Algo:

1. Use fast & slow ptr until they meet
2. Start another slow ptr at the head & move slow & slow2 one step at a time until they meet.

The meeting point of slow & slow2 is the start of the cycle



$$1 \leq k \leq n$$

$$0 \leq p < k$$

We do s iterations of slow ptr

$$s = L + tk + p$$

After ' s ' iterations, how far from the fast be?



$$= 2s$$

$$= 2[L + tk + p] \quad \underline{L + ck + p}$$

$$= 2L + 2tk + 2p$$

$$= L + \underline{L + p + 2tk} + p$$

if this part is a multiple of k , then they meet

$$L + p + 2tk = \text{multiple of } k$$

$$\Rightarrow \underline{L + p \text{ has to be a multiple of } k}$$

$$L + p = ck$$

$$\Rightarrow L = ck - p$$

$$= \underbrace{(c-1)k + k}_{ck} - p$$

$$\boxed{L = (c-1)k + (k-p)}$$

State Machines

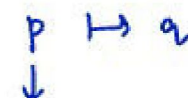
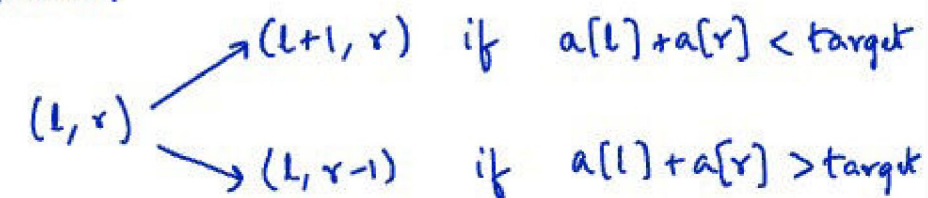
- States: $\{q\}$
 - Initial state: q_0
 - Transitions: $\{q \rightarrow r\}$
 - Reachability:
 - A state is reachable from the initial state if there exists a path from the initial state through valid transitions
 - Invariant:
 - A property that holds true for every reachable state.
 - state is reachable $\Rightarrow$ Invariant is true for that state
- If invariant is false $\Rightarrow$ not reachable

• Sorted 2-sum:

• State: $\{(l, r)\}$

• Initial state: $(0, n-1)$

• Transitions:



• $a[l] + a[r] < \text{target}$

• solution is contained in $[l, r]$

$\begin{cases} a[l] + a[r] < \text{target} \\ a[l] + a[i] < \text{target} \\ \text{for all } i \text{ in } [l+1, r-1] \end{cases}$

$\swarrow$

l is not part of the solution
 $\therefore$ we can increment 'l'

Number Representations

$$758 = 700 + 50 + 8$$

$$00758 = \underline{7} \cdot 10^2 + \underline{5} \cdot 10^1 + \underline{8} \cdot 10^0$$

Base 10 $\Rightarrow$ digits: 0, 1, 2, ..., 9

Base 2 $\Rightarrow$ " : 0, 1

Base 3 $\Rightarrow$ " : 0, 1, 2

Base 16 $\Rightarrow$: 0, 1, 2, ..., 10, 11, 12, 13, 14, 15
 $\downarrow$
 Hexadecimal

	A	B	C	D	E	F
0x9DEF						

$$0x12 = 1 \times 16^1 + 2 \times 16^0$$

$$= 18_{10}$$

• Binary to decimal

$$\begin{array}{cccccc} 5 & 4 & 3 & 2 & 1 & 0 \\ 1 & 1 & 0 & 0 & 1 & 0 \end{array}$$

$$110010 = 1 \cdot 2^5 + 1 \cdot 2^4 + 0 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0$$

$$= 32 + 16 + 2 = \underline{50}$$

• Decimal to binary

50	1	1	0	0	1	0
	5	4	3	2	1	0

$$\begin{array}{l} 50 \\ \downarrow -2^5 \\ 18 \\ \downarrow -2^4 \\ 2 \\ \downarrow -2^1 \\ 0 \end{array}$$

• Decimal to base 3

(315)	1	0	2	2	0	0	$3^0 = 1$
	5	4	3	2	1	0	$3^1 = 3$
$\downarrow -243$							$3^2 = 9$
72							$3^3 = 27$
$\downarrow -2 \cdot 27$							$3^4 = 81$
18							$3^5 = 243$
$\downarrow -2 \cdot 9$							
0							

• Binary & Hexadecimal

0 - 0000	8 - 1000
1 - 0001	9 - 1001
2 - 0010	10(A) - 1010
3 - 0011	11(B) - 1011
4 - 0100	12(C) - 1100
5 - 0101	13(D) - 1101
6 - 0110	14(E) - 1110
7 - 0111	15(F) - 1111

$$0x67 = 1100111$$

$$= 0 \cdot 2^7 + 1 \cdot 2^6 + 1 \cdot 2^5 + 0 \cdot 2^4 + 0 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0$$

$$= 16^1 (2^4) (0 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0)$$

$$+ 16^0 (2^0) (0 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0)$$

Integer representations

- int - 32 bits
- short - 16 bits
- long - 64 bits

unsigned & signed
 ↓
 we have only non-negative values
 ↓
 we represent +ve & -ve values & 0

$$\begin{array}{r} 3 \ 2 \ 1 \ 0 \\ 1 \ 1 \ 0 \ 1 \\ \hline -1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 \end{array}$$

4 bits:

$$\begin{array}{r} 1111 = 15 \\ + \quad 1 \\ \hline 10000 = 0 \\ \times \end{array}$$

0	0	0	0
0	0	0	1
0	0	1	0
0	0	1	1
0	1	0	0
0	1	0	1
0	1	1	0
0	1	1	1
1	0	0	0
1	0	0	1
1	0	1	0
1	0	1	1
1	1	0	0
1	1	0	1
1	1	1	0
1	1	1	1

Unsigned int

0
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15

Sign & magnitude

0
1
2
3
4
5 ✓
6
7
-0
-1
-2
-3
-4
-5
-6
-7 ✓

One's complement

0
1
2
3
4
5
6
7
-7
-6
-5
-4
-3
-2
-1
-0

Two's complement

0
1
2
3
4
5
6
7
-8
-7
-6
-5
-4
-3
-2
-1

k bits → range of unsigned values
 $[0, 2^k - 1]$

unsigned int (32 bits): $[0, 2^{32} - 1]$

unsigned long (64 bits): $[0, 2^{64} - 1]$

$$\begin{array}{lcl} 7 & \rightarrow & 0111 \quad +1 \\ -8 = & 8 & \rightarrow 1000 \\ -7 = & 9 & \rightarrow 1001 \quad +1 \\ -6 = & 10 & \rightarrow 1010 \quad +1 \end{array}$$

$$\begin{array}{r} 11 \\ -7 \quad 1001 \\ +3 \quad 0011 \\ \hline -4 \quad 1100 \\ 7 \quad 0111 \\ 3 \quad 0011 \\ \hline 1010 \end{array}$$

4 bits $\rightarrow$ Unsigned range: $[0, 15]$
 $\hookrightarrow 2^4 - 1$
 $\rightarrow$ Signed range: $[-8, 7]$

32 bits $\Rightarrow$ representation of smallest #: $\overset{31}{1} \overset{30}{0} \dots \overset{27}{0} \overset{1}{0} \overset{0}{0} = -2^{31}$
representation of largest #: $0 \overset{31}{1} \dots \overset{1}{1} \overset{0}{1} = 2^{31} - 1$

$$[-2^{31}, 2^{31} - 1]$$

$$2^{30} + 2^{29} + \dots + 2^1 + 2^0$$

16 bits: $[-2^{15}, 2^{15} - 1]$
 $[0, 2^{16} - 1]$

$$\frac{1(2^{31} - 1)}{2 - 1} = 2^{31} - 1$$

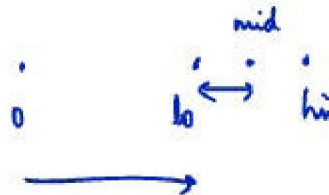
int - 4 bits $\rightarrow [-8, 7]$

$lo = 5$ $hi = 7$

$mid = (lo + hi) / 2$
 $\hookrightarrow mid = 6$

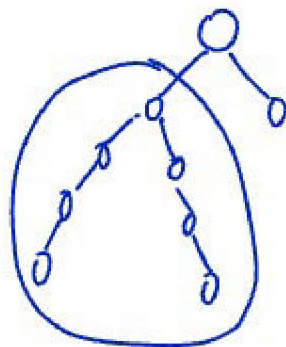
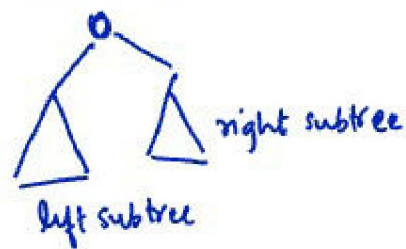
$= (5 + 7) / 2 = 12$

$mid = lo + \frac{(hi - lo)}{2}$



Trees

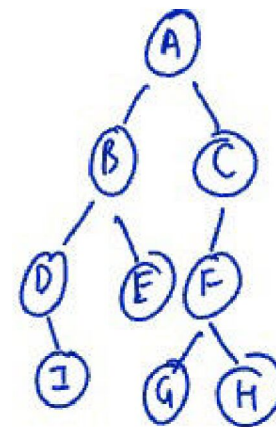
- Most of the questions can be solved using recursion



recursive function

→ wrapper fn
 def binarySearch(nums, key):
 return binarySearchR(nums, key, 0, len(nums)-1)

def binarySearchR(nums, key, lo, hi):
 mid:
 bsr(nums, key, lo, mid-1)
 bsr(" ", mid+1, hi)
 return mid



Preorder

A B D E C F G H
 - A(pre(B)) pre(c)
 - A(B pre(D) pre(E) pre(C))

Postorder



Pre order

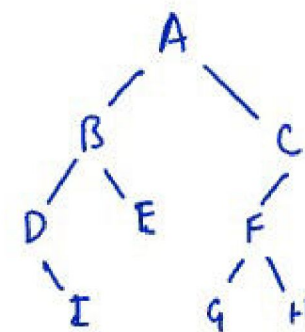
↓
 process node first then subtrees

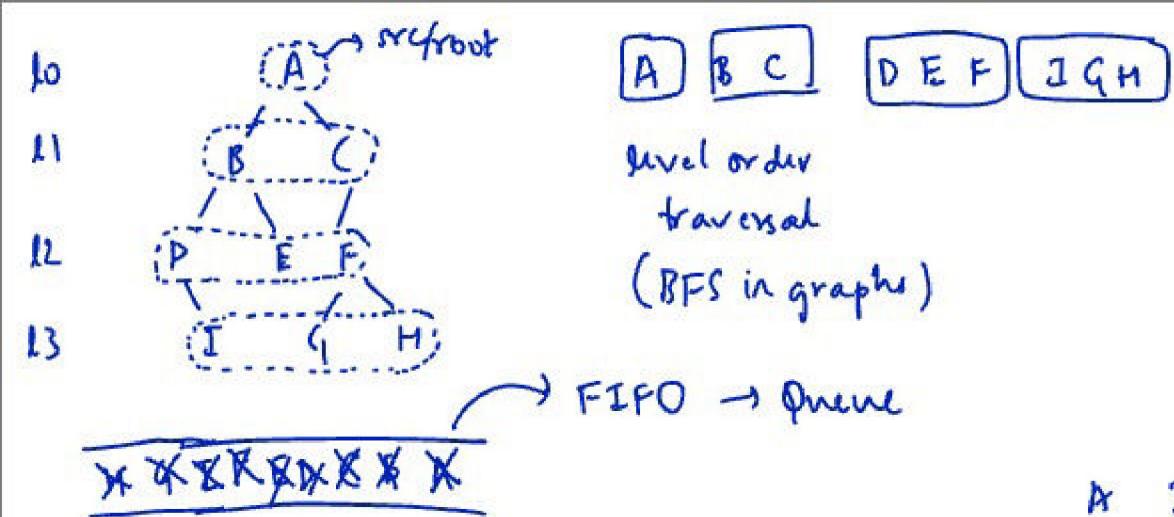
Post order

↓
 process subtrees first then node

in order

process left subtree
 process node
 process right subtree





as long as queue is not empty:

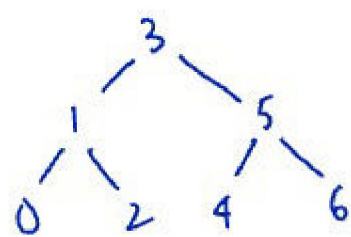
remove element from queue

(process it)

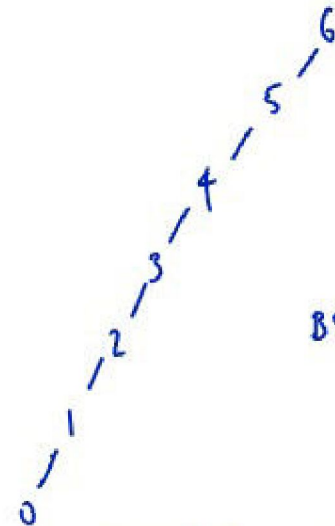
add its children from l to r into the queue

BST- Binary Search Trees

- assuming no duplicates
- BST property
 - for every node, all elements in its left subtree $<$ node's value
 - all elements in its right subtree $>$ node's value
- find(), add(), delete(), ...
 - $O(h)$ $\hookrightarrow$ height of the tree
 - for balanced BSTs, it is $O(\log n)$



BST? ✓

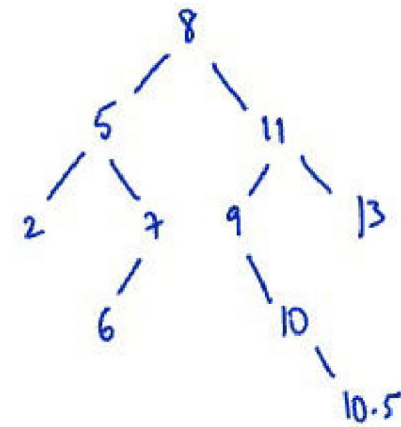


BST? ✓

Balanced binary trees

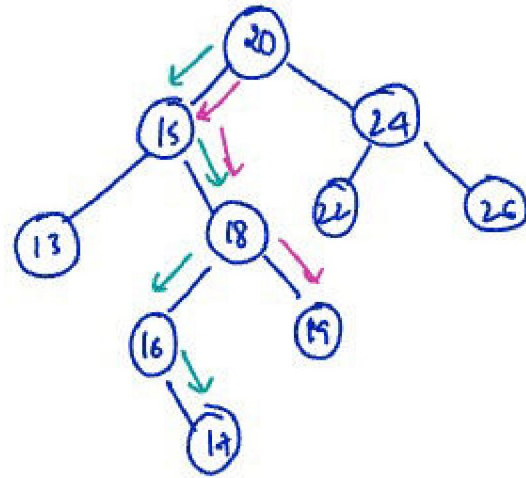
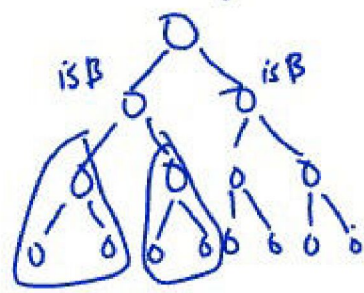
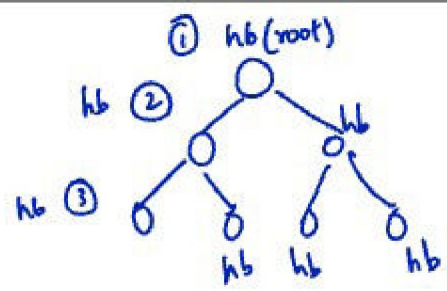
iff $h = \Theta(\log n)$

AVL trees (height balanced - diff between h of subtrees ≤ 1 for all nodes)
Red Black Trees etc

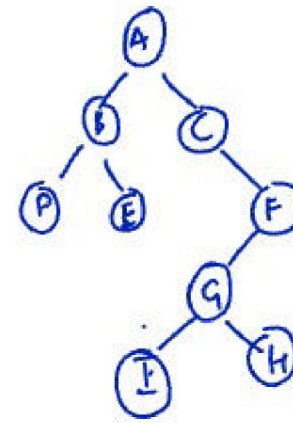


Inorder: 2 5 6 7 8 9 10 10.5 11 13

↓
of BSTs gives us the sorted order of values



LCA(17, 19)



right-nodes = []

queue
* →

while (!queue.isEmpty()):
// queue will exactly have
nodes from one level
level-nodes = list(queue)
for loop for len(q) times
process (node)

deque

↓
double ended queue in O(1)

list[a:b]
↑ inclusive
↓ not inclusive
length of slice = b - a

$i=10$ $i=10$
 $\downarrow$
 1 2 3 4 (

$res = 0$

$$res = res * 10 + 1 = 1$$

$$res = res * 10 + 2 = 12$$

$$res = res * 10 + 3 = 123$$

$$res = res * 10 + 4 = 1234$$

Horner's
method

$$ax^3 + bx^2 + cx + d \rightarrow 1 \cdot 10^3 + 2 \cdot 10^2 + 3 \cdot 10^1 + 4 \cdot 10^0$$

$$(((a)x + b)x + c)x + d$$

... (112 () (right tree)) ...

112

$$\Theta(n^d) \Rightarrow a_n x^n + a_{n-1} x^{n-1} + a_{n-2} x^{n-2} + \dots + a_2 x^2 + a_1 x + a_0$$

$$3x^2 - 5$$

$$4x^{12}$$